End-to-End Machine Learning Project

Part 8: select and train a model – fine-tune the system

Marcus Birkenkrahe

COSC 3430 · Artificial Intelligence · Augustana University

Based on Géron, *Hands-On Machine Learning with Scikit-Learn and PyTorch*, Ch. 2

Fall 2026

Colab starter notebook: `https://birkenkrahe.github.io/go/select-train`

Roadmap

Ten steps, over several code-along lectures:

1. Look at the big picture
2. Get the data
3. Explore and visualize the data
4. Create the test data
5. Prepare the data for learning ✓
6. Select a model and train it
7. Fine-tune the model
8. Present your solution
9. Launch the production system
10. Monitor and maintain it

Today: step 6, select a model and train it – and a first look at step 7, fine-tune the model.

Where part 7 left off

- Problem framed ✓
- Data obtained ✓
- Data explored ✓
- Training and test set sampled ✓
- Preprocessing pipeline built ✓: `preprocessing` turns the 9 columns of `housing` into 24 features, ready for training

How do you know which model to pick?

You don't know it in advance.

You can go by experience with a domain (the data, or other models that worked), or by what you see (the data look as if a linear function would approximate them).

But you don't know – so you don't commit to one model in advance.

Why linear regression first?

A cheap **baseline**:

- No hyperparameters to tune, trains instantly
- Fully interpretable: read the effect of each feature straight off its coefficient
- Tells you something about the data: do the features act linearly?
- If it does not work, the relationships are more complex (interactions, non-linearities) – a reason to try e.g. decision trees

The model selection process

1. Quickly try a handful of model families (linear, tree-based, ensemble), without spending much time on their hyperparameters
2. Compare them with **cross-validation** – not the test set!
3. Shortlist two to five promising candidates
4. Fine-tune their hyperparameters

No free lunch theorem (Wolpert, 1996)
: no single model is universally best across all possible datasets. There is no analytical shortcut to "the right model" – it is picked empirically.

Train a linear regression

→ **Switch to editor · code-along 1**

```
from sklearn.linear_model import LinearRegression

lin_reg = make_pipeline(preprocessing, LinearRegression())
lin_reg.fit(housing, housing_labels)

housing_predictions = lin_reg.predict(housing)
print(housing_predictions[:5].round(-2))
print(housing_labels.iloc[:5].values)
```

```
[246000. 372700. 135700.  91400. 330900.]
[458300. 483800. 101700.  96100. 361800.]
```

Some predictions are close (91,400 for 96,100), others far off (246,000 for 458,300).

RMSE on the training set

→ **Switch to editor · code-along 2**

```
from sklearn.metrics import root_mean_squared_error

lin_rmse = root_mean_squared_error(housing_labels, housing_predictions)
print(round(lin_rmse))
```

```
68973
```

A typical prediction is off by about **$69,000** – in dollars, like the target.

Why RMSE, and not `score()`?

`score()` returns R^2. Why do we use RMSE instead?

Recall: $R^2 = 1 - \dfrac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2} = 1 - \dfrac{\mathrm{RMSE}^2}{\mathrm{Var}(y)}$ here: $1 - \dfrac{68{,}973^2}{115{,}311^2} = 0.64$

1: perfect 0: no better than predicting the mean $\bar{y}$ < 0: worse

- $\rightarrow$ RMSE comes out in **dollars**
- $\rightarrow$ R^2 is relative to how much the target varies in *this* dataset: a high R^2 can go with a large dollar error. Hard to compare across datasets.
- $\rightarrow$ Consistent with what we did so far: an absolute metric is easier to interpret than a relative one

Underfitting

- Most districts' `median_house_value` lies between **$119,000** and **$264,000** (25th and 75th percentile)
- A typical error of **$69,000** is not satisfying: the model **underfits** – the features do not carry enough information, or the model is not powerful enough

Countermeasures:

1. Select a more powerful model
2. Feed the training algorithm better features
3. Reduce the constraints on the model (less regularization) – `LinearRegression` is not regularized, so this option is out

A decision tree

→ **Switch to editor · code-along 3**

```
from sklearn.tree import DecisionTreeRegressor

tree_reg = make_pipeline(preprocessing,
                         DecisionTreeRegressor(random_state=42))
tree_reg.fit(housing, housing_labels)

housing_predictions = tree_reg.predict(housing)
tree_rmse = root_mean_squared_error(housing_labels, housing_predictions)
print(tree_rmse)
```

```
0.0
```

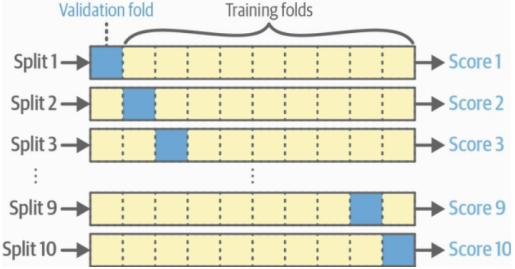
A decision tree predicts by a series of yes/no questions (splits on feature thresholds) until it reaches a leaf that gives the prediction. It finds nonlinear relationships.

Perfect, or overfitting?

- An error of **0**: either the model is perfect, or it **overfits** badly – it has learned every training district by heart
- You cannot be sure until you run it on unseen data
- But touch the test set only with a model you trust: otherwise you use the test score to make decisions, and it no longer tells you how the model does on new data
- Instead: split off a **validation set** from the training set – or use **cross-validation**

k-fold cross-validation

- Split the training set into k non-overlapping **folds**
- Train and evaluate the model k times: each time, evaluate on a different fold and train on the other $k - 1$
- Result: k evaluation scores – a mean *and* a spread
- The test set stays untouched



Cross-validate the tree

→ **Switch to editor · code-along 4**

```
from sklearn.model_selection import cross_val_score

tree_rmses = -cross_val_score(
    tree_reg, housing, housing_labels,
    scoring="neg_root_mean_squared_error",
    cv=10)
print(pd.Series(tree_rmses).describe())
```

```
count       10.000000
mean     66573.734600
std       1103.402323
min      64607.896046
25%      66204.731788
50%      66388.272499
75%      66826.257468
max      68532.210664
```

Cross-validation expects a **utility** function (greater is better), so the score is the *negative* RMSE. The minus sign in front turns it back into the RMSE.

What the cross-validation results mean

Each RMSE: a new tree trained on 9 folds, scored on the 10th, which it has not seen

Row	Value	Meaning
`count`	10	`cv=10`: 10 RMSEs
`mean`	66,574	error on unseen data
`std`	1,103	folds differ a little
`min`	64,608	best fold
`25%`	66,205	1st quartile
`50%`	66,388	median $\approx$ mean
`75%`	66,826	3rd quartile
`max`	68,532	worst fold

- On districts it did not train on, the tree is off by about **$66,600**
- All 10 folds lie within about $4,000: the estimate is **precise**. One validation set gives one number and no spread.
- Training RMSE 0, validation RMSE $66,600: the tree **overfits**

Cross-validate the linear regression

→ **Switch to editor · code-along 5**

```
lin_rmses = -cross_val_score(
    lin_reg, housing, housing_labels,
    scoring="neg_root_mean_squared_error",
    cv=10)
print(pd.Series(lin_rmses).describe())
```

```
count       10.000000
mean     70003.404818
std       4182.188328
min      65504.765753
25%      68172.065831
50%      68743.995249
75%      70344.943988
max      81037.863741
```

Tree: 66,574 vs. 70,003 – slightly better, but nowhere near its training error of 0. The tree **overfits**.

Linear regression: 70,003 vs. its training error of 68,973 – close together, and both high. It **underfits**.

A random forest

→ **Switch to editor · code-along 6 – start it now, it takes several minutes**

```
from sklearn.ensemble import RandomForestRegressor

forest_reg = make_pipeline(
    preprocessing,
    RandomForestRegressor(random_state=42, n_jobs=-1))
forest_rmses = -cross_val_score(
    forest_reg, housing, housing_labels,
    scoring="neg_root_mean_squared_error",
    cv=10)
print(pd.Series(forest_rmses).describe())
```

```
count       10.000000
mean     47038.092799
std       1021.491757
min      45495.976649
25%      46510.418013
50%      47118.719249
75%      47480.519175
max      49140.832210
```

- Many decision trees on random subsets of the features, predictions averaged
- An **ensemble**: if the models are diverse, averaging smooths out their errors

Training error vs. validation error

Model	Training RMSE	Validation RMSE	
`LinearRegression`	68,973	70,003	underfits
`DecisionTreeRegressor`	0	66,574	overfits
`RandomForestRegressor`	17,551	47,038	better, still overfits

- Both errors high and close together: the model is too simple
- Training error far below validation error: the model memorizes, it does not generalize
- Next to try: support vector machines, neural networks, … – no free lunch: there is no shortcut past trying them

Practice: cross-validate a k-nearest neighbors regressor

In your notebook, section **Practice**:

1. Put a `KNeighborsRegressor` (default hyperparameters) behind `preprocessing`
2. Run 10-fold cross-validation
3. Compare its mean RMSE with the three models. Where does it land?

`LinearRegression`	70,003
`DecisionTreeRegressor`	66,574
`KNeighborsRegressor`	**60,328**
`RandomForestRegressor`	47,038

Same `preprocessing`, one line changed: kNN beats the linear regression and the tree, but not the forest.

Fine-tune the system

- With a shortlist of promising models, adjust their **hyperparameters** – settings you choose before training: the maximum depth of a tree, the number of trees in a forest, the number of neighbors in kNN
- `GridSearchCV` cross-validates every combination of the values you give it
- `RandomizedSearchCV` tries a fixed number of random combinations – better when the search space is large
- Or combine the best-performing models into an **ensemble**

Does this make ML a black box?

With all these model combinations, is ML not almost a black box?

- → The **pipeline** stays transparent: every transformation is explicit, named and inspectable
- → The **model** gets harder to read as it gets more complex: `LinearRegression` – read the coefficients; `DecisionTreeRegressor` – follow the splits; `RandomForestRegressor` – an average of many trees, no single "reason" for a prediction
- → A tradeoff: flexible models fit more of the pattern, but are harder to audit. Tools such as `feature_importances_`, permutation importance and partial dependence plots open the box after the fact.
- → The more powerful the model, the more trust has to come from **validation**, not from reading the model

What we learned

- **Start simple**: linear regression is a cheap baseline – its training error shows whether you underfit
- **Training error alone misleads**: a tree with training error 0 overfits – compare training and validation error
- **Cross-validation**: k validation scores from the training set alone, the test set stays untouched
- **Ensembles** such as random forests do better, but are harder to read
- **No free lunch**: the best model is found by trying, not by theory

Next: back to classification – the **confusion matrix**. Fine-tuning and the final evaluation on the test set: read them in the textbook. Géron, Ch. 2, "Fine-Tune Your Model"